

Aberto: terça-feira, 4 nov. 2025, 13:00

Vencimento: segunda-feira, 24 nov. 2025, 23:59

EP-03 - Otimização de Ajuste de Curvas (OPCIONAL)

Objetivo

O objetivo deste trabalho é melhorar e avaliar o desempenho do programa de ajuste de curvas **ajustePol**, anexado neste enunciado. O programa calcula um polinômio de grau **G** que se ajusta a uma curva descrita por **P** pontos.

Especificação do Programa

O programa **ajustePol** calcula um ajuste de curva polinomial **f(x)** de grau **G** a partir de uma tabela de **P** pontos **(x,y)**.

São calculados os coeficientes **aj** do polinômio $f(x) = a_0 + a_1x + a_2x^2 + \dots + a_Gx^G$ utilizando-se do **Método dos Mínimos Quadrados**. O método da **Eliminação de Gauss com pivoteamento parcial** deve ser usado para resolver o sistema linear proveniente do método.

O programa recebe como entrada o grau **G** do polinômio de ajuste (1ª linha), a quantidade **P** de pontos da tabela (2ª linha) e a tabela com os pontos **(x,y)** do tipo **double**, um par de coordenadas por linha.

O programa mostra na saída os coeficientes **aj** da equação de ajuste bem como os resíduos entre a tabela inicial de pontos e os valores produzidos pela equação calculada (**ri = | yi - f(xi) |**).

A última linha da saída mostra o valor de **K** seguido dos tempos gastos (em milisegundos) dos trechos medidos **tSL** (geração do SL) e **tEG** (solução do SL).

Entrada (via stdin):

G

P

x1 y1

?

```
x2 y2
```

```
...
```

```
xP yP
```

Saída (via stdout):

```
a0 a1 a2 ... aG
```

```
r0 r1 r2 ... rP
```

```
P tSL tEG
```

O programa deve ser invocado da seguinte forma:

```
./ajustePol < pontos.in > resultado.out
```

Melhoria de Desempenho

Você deve alterar o código do programa **ajustePol** original (**v1**) de forma a obter uma melhora no desempenho (**v2**).

ATENÇÃO: as alterações efetuadas **não devem** alterar o resultado do programa, exceto por erros numéricos. Isto é, as alterações devem otimizar o programa sem prejudicar o seu funcionamento.

As alterações no código, seja na geração do SL, na solução do SL, nas estruturas de dados ou qualquer outro ponto relevante, devem ser **explicadas** no LEIAME a ser entregue, **justificando** as razões pelas quais você efetuou cada alteração.

Análise de Desempenho

Uma vez alterado o código, você deve comparar o desempenho das duas versões em relação a: **(A)** geração do SL pelo Método dos Mínimos Quadrados e **(B)** solução do SL pelo Método da Eliminação de Gauss.

Estas análises devem ser descritas sob a forma de gráficos.

É imprescindível que sejam respeitadas as seguintes condições:

1. Ambos códigos devem ser compilados com GCC e as opções: **-O3 -mavx -march=native**;
2. Os códigos devem ser compilados na mesma máquina utilizada para os testes;
3. Os testes devem utilizar os mesmos parâmetros e em igualdade de condições;
4. Ambos códigos devem ser instrumentados com a biblioteca do **LIKWID** e executados na **core** mais superior (no caso do DINF, na core **3**). Use a opção **-C <core>** ao executar o programa via **likwid-perfctr**;
5. Você pode escolher um computador de sua preferência, desde que possua os **?** contadores Likwid especificados. Não utilize as servidoras de processamento do DInf

que tenham uso compartilhado. Elas podem ser máquinas virtuais e o compartilhamento impede medidas de desempenho. **Em caso de dúvida, consulte o professor;**

6. Você deve apresentar a arquitetura do processador utilizado nos testes no seu texto. Estas informações podem ser obtidas com o comando "**LIKWID-topology -g -c**".

Para comparar o desempenho dos códigos, você deve efetuar uma série de testes.

- **Os testes** devem ser reportados sob a forma de **dois** conjuntos de gráficos de linhas sendo cada gráfico referente a um teste: um conjunto para o item **(A)** (geração do SL pelo Método dos Mínimos Quadrados), e um para o item **(B)** (solução do SL pelo Método da Eliminação de Gauss);
- **Os testes** devem ser executados para os seguintes valores de **G**: $G_1=10$ e $G_2=1000$. Para todos os valores de **G**, usar os seguintes valores de **P**: 64, 128, 200, 256, 512, 600, 800, 1024, 2000, 3000, 4096, 6000, 7000, 10000, 50000, 10^5 . Para **G1** acrescentar os seguintes valores de **P**: 10^6 , 10^7 , 10^8 ;
- **Cada gráfico** deve ter 4 (quatro) linhas em cores distintas para os resultados de cada combinação versão+grau (**G1+v1**, **G1+v2**, **G2+v1** e **G2+v2**);
- No eixo das **abcissas** os gráficos representam o número de pontos **P**.
 - Para gerar os dados de teste, compile o programa **gera_entrada** que gera **P** pontos para um ajuste de curva polinomial de grau **G** (vide Arquivos auxiliares abaixo) e use-o em pipeline com seu programa (onde **<P>** é a quantidade de pontos a ser gerado e **<G>** é o grau do polinômio):

```
./gera_entrada <P> <G> | ./ajustePol (sem LIKWID)
./gera_entrada <P> <G> | likwid-perfctr -C 3 -g <metrica> -m ./ajustePol (com LIKWID)
```

Por exemplo, para testar o ajuste para 100 pontos com um polinômio de grau 5, execute:

```
./gera_entrada 100 5 | ./ajustePol
```

- **ATENÇÃO:** em função dos tamanhos muito grandes de **P**, algumas variáveis usadas no programa **ajustePol** deverão ser do tipo **long long int**. Essa modificação deve ser feita em ambas versões (**v1** e **v2**), a fim de permitir que os testes sejam realizados.
- **Cada gráfico** deve ser explicado e você deve demonstrar que consegue entender o ? que está reportado nele;

- Os gráficos devem ser apresentados com o eixo das **abscissas** em **escala logarítmica**.

Os seguintes testes devem ser executados:

1. **Teste de tempo:** mostrar o tempo de execução de cada item monitorado usando **timestamp()**. No gráfico, esta **ordenada** deve estar também em **escala logarítmica**;
2. **Cache miss L3:** utilizar o grupo **L3CACHE** do LIKWID, e apresentar o resultado de **"cache miss RATIO"**;
3. **Operações aritméticas:** utilizar o grupo **FLOPS_DP** do LIKWID e reportar **FLOPS DP** e **FLOPS AVX DP**, em **MFLOP/s**.

No arquivo LEIAME devem ser explicados os resultados obtidos em FLOPS AVX DP;

Produto a ser entregue

Deve-se entregar um pacote de software completo contendo os fontes em **linguagem C** do programa original (**v1**), com as modificações introduzidas (**v2**), um arquivo LEIAME, Makefile, os scripts de automatização dos testes, as tabelas geradas a partir do LIKWID e os gráficos gerados a partir destas tabelas, com explicação dos resultados no arquivo LEIAME.

O pacote deve ser arquivado e compactado com **zip** ou **tar**, em um arquivo chamado **login1.<ext>**, onde **login1** é o login (nos sistemas do DINF) do aluno autor do trabalho. O pacote deve ter a mesma estrutura de diretório e arquivos dos trabalhos anteriores. Lembre-se que **<ext>** é **tar**, **tar.gz** ou **zip**.

Arquivos auxiliares

Abaixo encontra-se a verso **v1** do programa **ajustePol** (que dever ser otimizada em uma verso **v2**), bem como o programa **gera_entrada**.

Para implementação de funções auxiliares (como **timestamp()**) e scripts para geração de gráficos, pode-se usar o git abaixo (pastas **utils** e **gnuplot**):

<https://gitlab.c3sl.ufpr.br/nicolui/ci1164-utils>

Para geração de dados de entrada, compile e use o programa **gera_entrada.c** abaixo conforme indicado anteriormente. Para compilar use:

```
gcc gera_entrada.c -o gera_entrada
```



[ajustePol.c](#)

3 novembro 2025, 12:27 PM



[gera_entrada.c](#)

3 novembro 2025, 12:27 PM

-  [Makefile](#)
-  [utils.c](#)
-  [utils.h](#)

3 novembro 2025, 12:27 PM
31 julho 2025, 14:46 PM
3 novembro 2025, 11:13 AM

Status de envio

Status de envio	Nenhum envio foi feito ainda
Status da avaliação	Não há notas
Tempo restante	A tarefa está atrasada há: 208 dias 14 horas
Última modificação	-
Comentários sobre o envio	> Comentários (0)

?